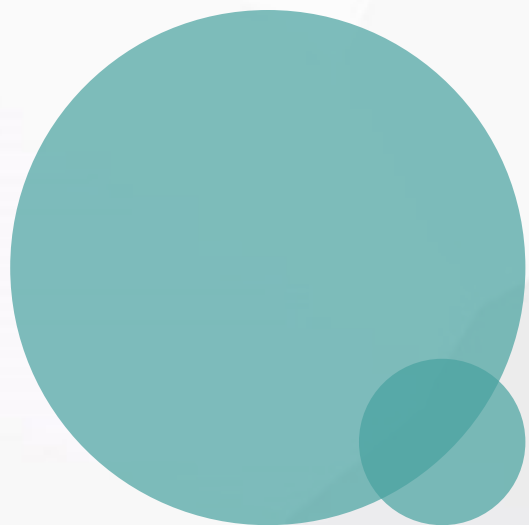




位运算

按位与 & (and)

与操作：

- $1 \& 1 = 1$
- $1 \& 0 = 0$
- $0 \& 1 = 0$
- $0 \& 0 = 0$

按位与：对每一个二进制位进行与操作

例： $21 \& 19 = 17$

- $21 = (10101)_2$
- $19 = (10011)_2$
- $17 = (10001)_2$

如何利用与运算，判断 x 是否能被 2 整除？

按位或 | (or)

或操作：

- $1|1 = 1$
- $1|0 = 1$
- $0|1 = 1$
- $0|0 = 0$

按位或：对每一个二进制位进行或操作

例： $21 | 19 = 23$

- $21 = (10101)_2$
- $19 = (10011)_2$
- $23 = (10111)_2$

按位异或 $\oplus$ (xor)

异或操作：

- $1 \oplus 1 = 0$
- $1 \oplus 0 = 1$
- $0 \oplus 1 = 1$
- $0 \oplus 0 = 0$
- 相同为 0，相异为 1

按位异或：对每一个二进制位进行异或操作

例： $21 \oplus 19 = 6$

- $21 = (10101)_2$
- $19 = (10011)_2$
- $6 = (00110)_2$

C++ 可以使用 $\wedge$ 表示异或

按位异或 $\oplus$ (xor)

异或的性质：

- $0 \oplus a = a$
- $a \oplus a = 0$
- $a \oplus b = b \oplus a$
- $(a \oplus b) \oplus c = a \oplus (b \oplus c)$
- 已知 $a \oplus b = c$
- 如果我们知道了 a 和 c ，求 b ？

左移 << 右移 >>

顾名思义，在二进制意义下整体左移或者右移，移动产生的新位置补 0。

例： $5 \ll 2 = 20$, $5 \gg 2 = 1$

- $20 = (10100)_2$
- $5 = (00101)_2$
- $1 = (00001)_2$
- 如果右移越过最右端？如果左移越过最左端？过界部分就没了
- 可以用来快速乘除 2 的幂次（位运算比加减乘除更加低级，速度更快）
 - $x \ll i$ 等价于 $x \times 2^i$
 - $x \gg i$ 等价于 $x \div 2^i$

位运算和逻辑运算的区别

- 逻辑运算是 `bool` 变量之间的运算。
 - 当接受整型数时，会将非零数作 `true` 转化成 `bool` 变量，做仅一位的位运算。
 - 其结果仍是 `bool` 类型。
- 位运算的结果是整型数的运算。
 - 它对每一个二进制位都进行运算。
 - 结果为整型数。

运算符的优先级（从高到低）

优先级	描述	运算符
1	括号	()、[]
2	正负号	+, -
3	自增自减，非	++, --, !
4	乘除，取余	*, /, %
5	加减	+, -
6	移位运算	<<, >>, >>>
7	大小关系	>, >=, <, <=
8	相等关系	==, !=
9	按位与	&
10	按位异或	^
11	按位或	
12	逻辑与	&&
13	逻辑或	
14	条件运算	?:
15	赋值运算	=, +=, -=, *=, /=, %=
16	位赋值运算	&=, =, <<=, >>=, >>>=

运算符的优先级

- 不建议背诵
- 在进行位运算时，每一步位运算打一个括号
- 当你的程序用到位运算时，且输出结果不对的时候，不妨看看你使用的位运算的优先级是否正确

位运算

- 如何将一个整数 `num` 的第 `x` 位设置成 1? `num |= 1 << (x - 1);`
- 如何将一个整数 `num` 的第 `x` 位设置成 0? `num &= ~ (1 << (x - 1));`
- 如何将一个整数 `num` 的第 `x` 位设置成翻转? `num ^= 1 << (x-1);`
- 如何判断一个整数 `num` 的第 `x` 位是否为 1? `(num >> (x - 1)) & 1`
- 如何计算一个整数 `num` 的二进制中 1 的个数?

位运算

- 如何计算一个整数 `num` 的二进制中 `1` 的个数？

```
int popcount(int num) {  
    int cnt = 0;  
    while (num != 0) {  
        if ((num & 1) == 1)  
            ++cnt;  
        num >>= 1;  
    }  
    return cnt;  
}
```

函数 (g++ 私货, 复杂度 $O(1)$): `__builtin_popcount(num)`

__builtin 系列函数

- `__builtin_popcount(x)` 变量在二进制下 1 的数量
- `__builtin_ctz(x)` 变量在二进制下末尾 0 的个数（传入 0 是 UB 的）
- `__builtin_clz(x)` 变量在二进制下前导 0 的个数（传入 0 是 UB 的）
- `__builtin_ffs(x)` 变量在二进制下最后一个 1 在第几位（从末尾往前算的第一个 1 的位置）
- `__builtin_parity(x)` 变量在二进制下 1 的个数的奇偶性（偶数为 0，奇数为 1），显著快于 `__builtin_popcount(x)`

这些都是 g++ 私货，传入变量为 `unsigned int` 类型，复杂度均为 $O(1)$

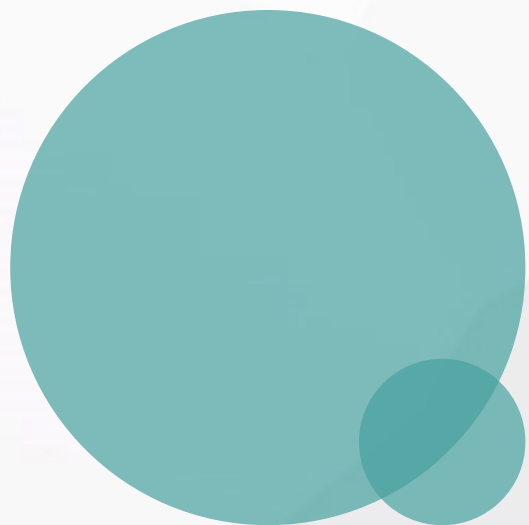
在这些函数的后面加上 `ll`（例如 `__builtin_popcountll`），就可以支持 `unsigned long long` 类型

洛谷 P1469 找筷子

有若干双筷子，筷子需要长度一样的才能组成一双。其中一只筷子被拿走了，给你剩下 n 只筷子的长度，问拿走的那只的长度。

$1 \leq n \leq 10^7 + 1$ 且为奇数， $1 \leq a_i \leq 10^9$

时限 2s，空间限制 4MB



枚举

枚举算法

- 枚举：基于已有知识来猜测答案的一种问题求解策略。
- 枚举的思想是不断地猜测，从可能的集合中一一尝试，然后再判断题目的条件是否成立。

枚举算法

- 枚举对象
 - 建立简洁的数学模型。可能的情况是什么？要枚举哪些要素？
- 枚举范围
 - 枚举的范围是什么？是所有的内容都需要枚举吗？
- 枚举的顺序
 - 根据题目判断。以怎样的顺序枚举更优、更可能在较短的时间内结束枚举？从小到大枚举更优还是从大到小枚举更优？
- 枚举的验证
 - 对于枚举到的解，如何判断它是否成立？

枚举算法

- 由 n 个数构成的数组 a 中的数互不相同。求其中和为 0 的数对的个数。
即求 (i, j) 的个数，满足
- 为整数。
- $|a[i]|, n \leq 10^6$ 。

枚举算法

- 枚举两个数的代码很容易就可以写出

```
for (int i = 0; i < n; ++i)
    for (int j = 0; j < n; ++j)
        if (a[i] + a[j] == 0)
            ++ans;
```

- 时间复杂度 $O(n^2)$

枚举算法

- 枚举范围的优化:
- 由于题中没要求数对是有序的，答案就是有序的情况的两倍
 - 考虑如果 (a, b) 是答案，那么 (b, a) 也是答案
 - 对于这种情况，只需统计人为要求有顺序之后的答案，最后再乘上 2
- 不妨要求第一个数要出现在靠后的位置

```
for (int i = 0; i < n; ++i)
    for (int j = 0; j < i; ++j)
        if (a[i] + a[j] == 0)
            ans += 2;
```

- 虽然复杂度相同，但在常数上节省了一半的开支

枚举算法

- 枚举的对象——两个数是否都一定要枚举出来呢？
- 枚举其中一个数之后，题目的条件已经确定了另一个数。
- 如果能找到一种方法直接判断题目要求的那个数是否存在，就可以省掉枚举后一个数的时间了
- $|a[i]| \leq 10^6$ ，且都是整数
- 我们可以使用桶记录遍历过的数

枚举算法

```
const int MAXN = 1e6;  
bool bucket[MAXN * 2 + 1]; // 初始是, bucket中元素均为false  
  
for (int i = 0; i < n; ++i) {  
    if (bucket[MAXN - a[i]]) ans += 2;  
    bucket[MAXN + a[i]] = true;  
}
```

- 时间复杂度 $O(n)$

二进制枚举

- 问题：给定一个大小为 n 的数集，判断该集合中存在多少个子集，满足子集中所有元素之和恰好为 0。 $(n \leq 20)$
- 如何枚举集合的一个子集？

二进制枚举

- 如何枚举集合的一个子集？
- 考虑一个大小为 n 的集合，它有 2^n 个子集
- 我们用一个 n 位的二进制数来表示选取状态
 - 这个数二进制第 i 位是 1 或 0，代表某个元素取或不取
- 比如集合是 $\{3, 2, 1, 4\}$
- $12 = (1100)_2$ 代表选择了 3 和 2，而不选择 1 和 4
- 0 至 $2^n - 1$ 这 2^n 个数恰好代表了这个集合的 2^n 种选取状态，不重不漏

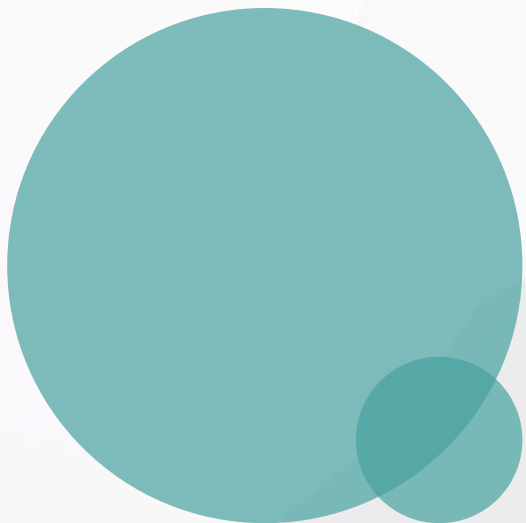
二进制枚举

- 给定一个大小为 n 的数集，判断该集合中存在多少个子集，满足子集中所有元素之和恰好为 0。 ($n \leq 20$)

```
int n, a[n], ans = 0;
for (int mask = 0, tot = (1<<n); mask < tot; mask++) {
    int sum = 0;
    for (int i = 0; i < n; i++) {
        if ((mask >> i) & 1) sum += a[i];
    }
    if (sum == 0) ans++;
}
```

二进制枚举

- 时间复杂度？
- 一共 2^n 个子集，对每个子集需要统计 n 个元素的状态。时间复杂度 $O(n2^n)$



一些常识 (二)

字典序，模运算

字典序

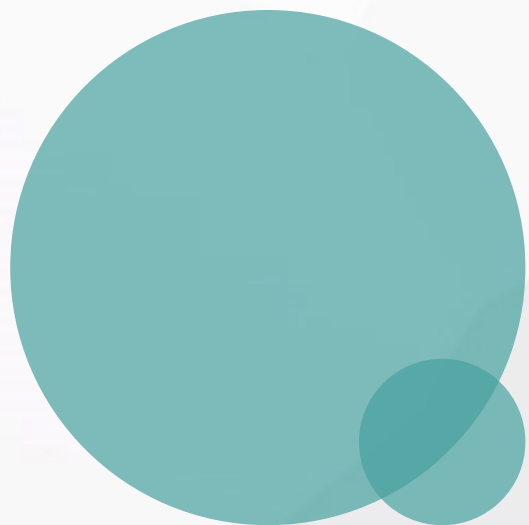
- 有两个序列 $\{a_i\}_{i=1}^n$ 和 $\{b_i\}_{i=1}^m$ ，称字典序 $a < b$ ，当且仅当存在一个下标 $i \in [1, \max(n, m)]$ ，满足 $a_i < b_i$ ，且对于任意的下标 $j \in [1, i)$ ，满足 $a_j = b_j$ 。若 x_i 不存在，则认为 x_i 无限小。
- 例如 $a = \text{"aabbcc"}$ ， $b = \text{"aacbbc"}$ ，则字典序（比较 ASCII 码） $a < b$
- 例如 $a = [1, 2, 1]$ ， $b = [1, 1, 6, 7]$ ，则字典序 $a > b$
- 例如 $a = [1, 1]$ ， $b = [1, 1, 1]$ ，则字典序 $a < b$
- 例如 $a = [1, 2, 3]$ ， $b = [1, 2, 3]$ ，则字典序 $a = b$

模运算

- 给定一个正整数 p ，任意一个非负整数 a ，一定存在等式 $a = kp + r$ ，其中 k, r 是整数，且 $0 \leq r < p$ 。仅存在唯一的 k, r 满足等式。
- 称 k 为 n 除以 p 的商， r 为 n 除以 p 的余数。
- 对于正整数 p 和非负整数 a ，定义取模运算： $a \% p$ 或 $a \bmod p$ ，表示 a 除以 p 的余数，即 $a \% p = a \bmod p = r$ 。
- 若 a 是负数，扩展定义： $a \bmod p = -(|a| \bmod p)$
 - 如果题意要求对整数取模，且没有规定整数正负，则默认取模后的结果是正整数。可以使用 $(a \bmod p + p) \bmod p$ 得到结果
- 若 p 是负数，扩展定义： $a \bmod p = a \bmod |p|$

模运算运算规则

- $(a \pm b) \bmod p = (a \bmod p \pm b \bmod p) \bmod p$
- $(a \times b) \bmod p = ((a \bmod p) \times (b \bmod p)) \bmod p$
- $a^b \bmod p = (a \bmod p)^b \bmod p$
- $a/b \bmod p$ 不一定等于 $a/(b \bmod p) \bmod p$
- $a/b \bmod p$ 不一定等于 $a \bmod p / b \bmod p$
- 除法取模需要用到数论中的逆元。
- $a^b \bmod p$ 不一定等于 $a^{b \bmod p} \bmod p$
- 指数取模需要用到数论中的欧拉降幂公式。



常见代码错误

只列举了一部分

运算结果溢出

- 在 C/C++ 中，`int` 类型的有效表示范围为 $[-2^{31}, 2^{31} - 1]$ 。在旧标准中并非如此，但现在都是这样。
- 当运算结果超出 `int` 范围时，可以考虑使用 `long long int` 类型，取值范围为 $[-2^{63}, 2^{63} - 1]$ 。
- 可以用 `typedef` 缩减代码长度。
- 当运算结果过大且没有取模时，应当考虑使用大整数运算。

```
int a = 2e9;  
int b = 2e9;  
int c = a + b;  
cout << c << endl;
```

```
long long a = 2e9 + 2e9;  
typedef long long ll;  
ll b = 1e18;
```


运算结果溢出

- 需要注意的是，两个 `int` 类型相乘，可能会超过 `int` 表示的范围。
- 可以参考右侧的写法。

```
int a=1e9, b=1e9;  
int c=a*b;  
/*(×)*/
```

```
int a=1e9, b=1e9;  
long long c=1ll*a*b;  
// 1ll 为 long long 类型字面量，  
大小为 1，第一步计算 1ll*a 的时  
候强转为 long long  
long long a=1e9,b=1e9;  
long long c=a*b;  
/*(√)*/
```

非法内存访问

- 非法内存访问会导致 Runtime Error, 即 RE, 包括但不限于:
 - 数组越界
 - 递归深度过大, 即递归爆栈
 - 对野指针取值、double free
- 但是数组越界并非一定会导致 RE, 有时会访问到其他变量上, 此时会很难 debug, 因此要格外小心。
- 在算法竞赛中少用指针。

```
#include <iostream>
using namespace std;
int a, b[2], c;
int main() {
    cin >> b[-1];
    cout << b[2];
    return 0;
}
```

大数组开成局部变量

- 在很多编译环境下，对局部栈空间的限制非常大，而竞赛题中经常用到 $1e5$ 乃至 $1e6$ 级别的数组，开成局部变量可能导致 CE 或 RE，此时应当将其开成全局变量。
- 局部变量的初值是随机的，全局变量的初值默认为 0
- `vector<int>` 等，分配在堆空间的 STL 容器可以放在局部变量

```
int main() {  
    int a[1000000]; // 6个0  
    return 0;  
} /*(X)*/
```

```
int a[1000000]; // 6个0  
int main() {  
    vector<int> a(1000000,0)  
    return 0;  
} /*(√)*/
```

IO 效率问题

- 在默认情况下，`cin/cout` 是极为迟缓的读入/输出方式
- 面对 10^6 数量级别的输入，用时达到 1s
- 可以在主函数开头加入语句 `ios::sync_with_stdio(false); cin.tie(0);`
- C++ 为了兼容 C，保证程序在使用了 `printf` 和 `cout` 的时候不发生混乱，将输出流绑定到了一起。同步的输出流是线程安全的。这也是使 `cin/cout` 速度较慢的主要原因。`ios::sync_with_stdio(false)` 关闭了同步的输入输出流
- 需要注意的是，执行该语句后不要同时使用 `cin/cout` 和 `scanf/printf`

IO 效率问题

- 在默认的情况下 `cin.tie()` 绑定的是 `&cout`，每次进行格式化输入的时候都要调用 `cout.flush()` 清空输出缓冲区，这样会增加 IO 负担
- 可以通过 `cin.tie(0)` 来解除绑定，进一步加快执行效率
- 需要注意的是，执行该语句后，程序中必须手动 `flush` 才能确保每次 `cout` 展现的内容可以在 `cin` 前出现（`cout << "Hello World" << flush`）。对于一般的算法竞赛题不需要手动 `flush`，因为输入是一次性读完的；对于交互题（`cin/cout` 交替出现），需要手动 `flush`。

IO 效率问题

```
int main() {  
    ios::sync_with_stdio(false);  
    cin.tie(0);  
    int a, b;  
    cin >> a >> b;  
    cout << a + b;  
}
```

IO 效率问题

- getchar 和 putchar 的输入输出效率极高
- 也可以通过 getchar 和 putchar 手写 IO 操作

```
inline int read(){
    int x=0,f=1; char ch=getchar();
    while(ch<'0' || ch>'9') {if(ch=='-')f=-1;ch=getchar();}
    while(ch>='0' && ch<='9') {x=(x<<3)+(x<<1)+(ch^'0');ch=getchar();}
    return x*f;
}

inline void write(int x) {
    static int sta[35]; int top = 0;
    do {sta[top++]=x%10,x/=10;} while(x);
    while(top) putchar(sta[--top]+48);
}
```

IO 效率问题

- 或者直接使用 `scanf/printf` 输入输出，效率也是较高的
- Java 语言标准 IO 操作更为缓慢，也请在网上查找、使用较为快速的 IO 操作

尽量不要使用 float

- 因为竞赛中的浮点数运算经常有精度要求，而 `float` 的精度很低，在很多情况下都是不够用的，所以索性就不要去使用它，用 `double`。

UB 行为

- 为啥不开 `-O2` 能 AC，开了 `-O2` 就 RE/TLE/WA（运行时错误、超时、答案错误）了？（`-O2` 是一种编译优化的选项，在 XJTUOJ 上默认开启）
- 萌新求助，为啥本地数据（与评测机评测的数据相同）能过，提交就全 RE/TLE/WA（运行时错误、超时、答案错误）了？
- 你的程序可能有 UB。未定义行为（Undefined Behavior, UB），是一类对程序无任何限制的行为。不同的编译器，不同的环境，不同的编译选项，会有不同的结果。
- 我们在写程序的过程中，应当尽量避免这种行为。

UB 行为列举

- 带符号整型算术溢出。下面的代码，若输入的数计算结果溢出了，在编译器的优化下（开启 `-O2`），可能会输出 “Not overflow!”
- （无符号整型溢出是有定义的，“自然溢出”，忽略高于最高位上的数）
- （例如，只有 2 位的无符号整型：10 和 11，相加为 01）

```
int main() {  
    int x; cin >> x;  
    if(x + 1 < x) cout << "Overflow!" << endl;  
    else cout << "Not overflow!" << endl;  
    return 0;  
}
```

UB 行为列举

- 谭浩强行为：无法确定的运算顺序。返回值什么都有可能。
- 在一条语句中，同时存在 `++x` 和 `x++`。

```
int main() {  
    int x=1;  
    cout << (x++ + ++x) << endl;  
    return 0;  
}
```

UB 行为列举

- 越界访问（由于访问结果是未定义的，程序评测的结果可能不止导致 RE（运行时错误），有可能变成 WA 和 TLE（超时、答案错误））
- 访问未初始化的非全局变量（由于访问结果是未定义的，程序评测的结果可能不止导致 WA（答案错误），有可能变成 RE 和 TLE（运行时错误、超时，由于非期望值在你的程序中导致的数组访问错误、死循环等等））
- 无可视副作用的无限循环。不影响运行结果的死循环，编译器可能直接优化掉。

UB 行为

- 在 clang 编译器中，启用编译选项 `-fsanitize=undefined`，可以在编译时检测未定义行为。

编译文件大小问题

- 一些评测机会对编译文件的大小做出限制。
- 在上面那段代码中，编译完成后，得到的文件会记录下整个 `a` 数组（由于 `a` 中的第一个元素 `1` 不是平凡的），导致文件过大。
- 在下面的代码中，不附初值，`a` 是平凡的，编译文件只记录下需要开这么大的空间的平凡的数组。

```
int a[1000000] = {1};  
int main() {  
    return 0;  
} /*(X)*/
```

```
int a[1000000];  
int main() {  
    a[0] = 1;  
    return 0;  
} /*(√)*/
```

size unsigned 类型问题

- STL 容器的 `size` 方法（例如：`vector<int> v; v.size()`），返回的是 `unsigned int` 类型。
- 对于下面的 `for` 循环，当大小为 0 的时候，由于 1 强制转换成 `unsigned`，同时 `unsigned` 类型的自然溢出，会导致数组越界

```
for (int i = 0; i <= v.size() - 1; ++i) v[i]
```

- 你可以这样写：

```
for (int i = 0; i + 1 <= v.size(); ++i) v[i]
```

- 若你编译时启用编译命令 `-Wall`，则会发现这一行提示了 `Warning`，显示 `int` 和 `unsigned int` 的隐形比较。

deque, stack, queue 空间问题

- 在算法竞赛题目里，别用或尽量少用 deque 和 stack 和 queue
- deque 在大数据范围下、在评测机中，空间大小会遇水膨胀，变大变粗，然后就 MLE（内存超限）了。
- stack 和 queue 的底层也是基于 deque。也少用。
- 在确定空间大小比较小、可以通过的情况，可以使用。
- 通常使用数组手写。
- 同时，时间常数也比较大。

编译器 Bug

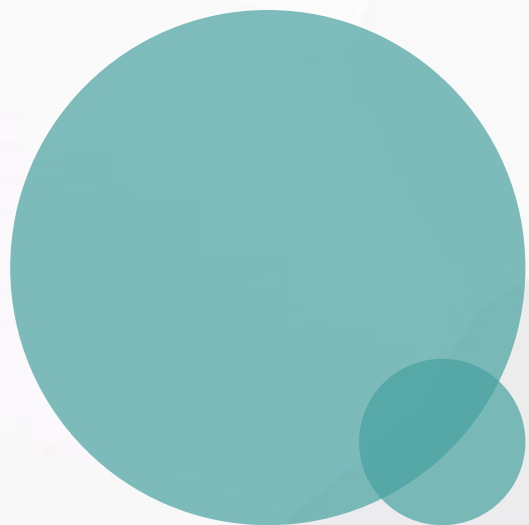
- 先别怀疑是编译器的问题。
- 若本地能过，评测机过不了，一般情况是你写了一些 UB 行为。
- 如果本地过不了，一般情况是你的代码逻辑有错误的。
- 编译器可能出问题，但编译器出问题不太可能。

sqrt 精度问题

- 对 `long long` 大小的数，尤其是 `1e18` 大小的数，`sqrt` 精度会不够。
- 对 `1e18` 的数，可能产生正负 10 的绝对误差。
- 这是由于 `sqrt` 传入的是 `double` 类型。
- 可以使用 `sqrtl`，传入的是 `long double` 类型，精度更高。
- 可以在 `sqrt` 得到的值附近枚举，得到正确的值

计算几何精度问题

- 以及一些浮点数类型的输出陷阱
- <https://www.cnblogs.com/zarth/p/7134442.html>



谢谢!